

Sharded RDBMS

1. What is Sharded RDBMS?

A **Sharded RDBMS** means splitting a relational database across multiple servers while still using SQL.

Instead of one huge SQL server:

None

Users Table → One MySQL/PostgreSQL Server

Use:

None

Shard 1 → DB Server A

Shard 2 → DB Server B

Shard 3 → DB Server C

Each shard stores part of relational data.

2. Why Use It?

Used when single SQL server hits limits:

None

Storage full

CPU bottleneck

Slow writes

Replication lag

Backup too slow

Too many concurrent users

But business still needs:

None

SQL queries

Transactions

Relational schema

Indexes

ACID features (limited depending architecture)

3. Common Platforms

- Amazon Web Services Amazon RDS
- MySQL with custom sharding
- PostgreSQL with partitioning/sharding tools
- Vitess
- Citus

4. Example

Suppose ecommerce orders table:

None

500 million rows

Shard by customer_id:

None

`customer_id % 4`

`0 → shard1`

`1 → shard2`

`2 → shard3`

`3 → shard4`

5. Sharding Strategies

A. Hash Sharding

None

```
hash(user_id) % N
```

Even distribution.

B. Range Sharding

None

```
1-1M users → shard1
```

```
1M-2M → shard2
```

Easy range queries.

C. Geographic Sharding

None

```
India → Mumbai DB
```

```
EU → Frankfurt DB
```

```
US → Virginia DB
```

D. Tenant Sharding

None

Enterprise A → shard A

Enterprise B → shard B

6. Biggest Challenge: JOIN Queries

Relational DB strength = joins.

But in sharded SQL:

None

orders on shard1

users on shard4

JOIN requires network communication.

7. Why JOIN Becomes Slow

Each node may need rows from others.

None

Shard A asks B,C,D

Shard B asks A,C,D

Heavy traffic + latency.

8. Best Practice for JOINS

Use same shard key for related tables.

Example:

None

`users sharded by customer_id`

`orders sharded by customer_id`

`payments sharded by customer_id`

Then user's related data lives together.

This is called:

None

`Co-location / Co-sharding`

Very important interview concept.

9. Aggregation Challenges

Simple query:

SQL

`SELECT SUM(amount) FROM orders;`

Needs partial sums from all shards.

Flow:

None

Shard1 sum = 100

Shard2 sum = 200

Shard3 sum = 300

App combines = 600

10. Easy Aggregations

Good candidates:

None

COUNT

SUM

AVG (sum + count)

MIN

MAX

11. Hard Aggregations

Difficult across shards:

None

MEDIAN

PERCENTILE

TOP-N global ranking

Complex window functions

Need more compute + data movement.

12. Transactions Become Harder

Single shard transaction:

None

Easy

Cross-shard transaction:

None

Complex

Needs:

- Two-phase commit
- Saga pattern
- Compensation logic

13. Advantages of Sharded RDBMS

None

Keep SQL ecosystem

Horizontal storage growth

Higher throughput

Can retain relational schema

Use existing SQL skills/tools

14. Disadvantages

None

JOIN slower

Cross-shard transactions hard

Operational complexity

Rebalancing difficult

Schema migrations harder

Global indexes complex

15. Good Use Cases

Use when:

- Need SQL + structured schema
- Large OLTP workloads
- Tenant/customer partitioning natural
- Moderate joins can be optimized

Examples:

- SaaS products
- Ecommerce orders
- Banking customer partitions
- CRM systems

16. Avoid When

Avoid if:

None

Heavy many-to-many joins

Frequent cross-entity joins

Need simple operations team

Huge analytics scans

Then maybe use:

- Data warehouse
- NoSQL
- Separate OLTP + OLAP design

17. Common Architecture Pattern

None

App Layer



Shard Router



Shard1 MySQL

Shard2 MySQL

Shard3 MySQL

Router decides destination shard.

18. Interview HLD Points

If proposing sharded RDBMS, mention:

None

Shard key selection

Rebalancing plan

Cross-shard joins minimized

Read replicas

Backup per shard

Global IDs

Migration strategy

19. Golden Rule

Choose shard key based on:

None

Most common query pattern

Write distribution

Data locality

Future growth

Wrong shard key causes major pain.

20. One-Line Summary

Sharded RDBMS horizontally scales SQL databases by splitting relational data across multiple nodes, but introduces challenges in joins, aggregations, transactions, and operational complexity.